Assignment 2 - Lists, Stacks and Queues

Course: COMP 8042

Instructor: Ailbhe Gill

Friday 2nd October, 2026

1 Introduction

The purpose of this lab is to develop your understanding of Lists, Stacks and Queues. You should be familiar with some of this material from previous courses. Submit your code and answers (answers typed or written by hand but legible) to the learning hub. This assignment is due on October 19th. You will be given time to work on it during the lab sessions.

2 Use of AI tools

Do not use any AI tools such as ChatGPT, Gemini, Copilot, etc. to generate answers. I will be asking you about your solutions, if you're not able to reproduce them (to a reasonable standard) or cannot talk me through them then I will assume you have used these tools. If you submit work that is not your own, that is plagiarism.

3 Grading

The following rubric will be generally used for problems where you need to submit code:

- Correctness of Implementation (65%).
- Clarity/Originality of code and discussion or justification (35%).

*For comments, there is no need to go overboard. Put a line or two of javadoc comments on class and methods stating what the method does. Make sure that any tricky or complex parts of the code are also explained (in your own words) in a comment.

4 Submission

Please submit a single pdf file with your code and answers for each of the 3 sections listed below. Submit your source code separately as individual .java files. You are welcome to use code provided by me (in the Content section on learning hub) as part of your solutions or as a starting point modified with your own code.

If you still need time to familiarize yourself with Java please email me and I can give you an exemption to use python for this assignment. If you don't email me with this request before Oct 8th you will not be able to get an exemption. All other assignments will require you to use Java (no exemptions).

5 List Problem - 10 marks

In this problem you are asked to design a list data structure for a specific task.

```
Interface Remover<T>{
    void removeEvens(yourEfficientList<T>);
    void removeOdds(yourEfficientList<T>);
    void removeDuplicates(yourEfficientList<T>, T duplicate);
}

class RemoverImpl<T> implements Remover<T>{
    /*
    Implements the set of in-place remove methods
    */
    void removeEvens(yourEfficientList<T>){
        // your code here
    }

    void removeOdds(yourEfficientList<T>){
        // your code here
    }

    void removeAllInstances(yourEfficientList<T>, T elementToRemove){
        // your code here
    }
}
```

1. - You are faced with a problem - you have been given a list of integers and need to remove all of the even elements as efficiently as possible. You know that removal is generally O(n) for the list implementations that we have studied. Create a new list class that has the capability to remove the even numbers more efficiently than calling ***remove*** repeatedly (multiple calls to an $O(n)$ method will result in an $O(k * n)$ algorithm, if k is the number of even elements in the list). **There are no constraints on how you design your data structure except that you cannot use the build-in list implementations. Marks will be awarded for reusability**.

   Hint: If the list is half odd, then we'll have $O(n \cdot \frac{n}{2}) = O(n^2)$ algorithm with the unmodified list classes. But you should be able to do this in $O(n)$. You may want to examine the iterator in the inner class of the list implementations provided in class. You may add methods to it.

   Note: You don't need to call your data structure class YourEfficientList, just implement the interface with whatever you name your list.

2. - Justify the decisions you made for your implementation in part 1 and state the Big-O runtime complexity of algorithm.

3. - Use the same (unmodified) data structure from part 1 to write a method which can remove odd numbers from the list. Demonstrate this working in a main() method, or write a JUnit test for it to prove it works as intended.

4. - Use the same (unmodified) data structure from part 1 to write a program to allow the user to

efficiently remove all instances of an element from a list. (i.e. if the list is [1,5,6,4,6,2,8,6], after running your algorithm with input 6, you'd end up with [1,5,4,2,8]). Demonstrate this working in a main() method, or write a JUnit test for it to prove it works as intended.

5. - Read about Java streams.

   (a) Use a java stream and the filter function to implement 1, 2 and 3 above with the built-in java ArrayList (java.util.ArrayList). What is the Big-O runtime for this implementation?

   (b) Write another method using streams, filters and the reduce method which filters the even numbers in a list, removes all instances of a given number and then sums the result. It's signature should be something like the following:

   int myFilter(List<Integer>, int elementToFilter);

Submit your data structure as 1 java file and your algorithms using this data structure in a second java file.

6 Stack Problem - 10 marks

1. Write a stack data structure using an ArrayList as the underlying container. It should be generic and obey the interface below:

```
1  public interface Stack<T> {
2      //Tests if this stack is empty.
3      boolean empty();
4
5      //Looks at the object at the top of this stack without removing it from the stack.
6      T peek();
7
8      //Removes the object at the top of this stack and returns that object as the value of this
       ↪  function.
9      T pop();
10
11     //Pushes an item onto the top of this stack.
12     T push(E item);
13
14 }
15
16 public class ArrayListStack<T> implements Stack<T>{
17     private ArrayList<T> list;
18
19     /*
20     Your code here
21     */
22 }
```

2. State the big-O complexity for the ***pop***, ***push*** and ***peek*** methods you have implemented.

3. Write a program which takes as input the text for a java program and matches parentheses within the program (i.e. you want to check if every '(', '{' and '[' has a corresponding ')', '}' and ']', respectively. You should use your stack data structure from part 1 to implement your algorithm.

4. Run your program on the java programs provided and document which programs do/don't have matching parentheses. If any opening parentheses don't have a corresponding closing parentheses, state the line number on which the first of these mismatches occurs.

5. Give the Big-O runtime complexity of your parentheses matcher algorithm.

7 Queue Problem - 10 Marks

Use the java LinkedList queue implementation found in java.util to solve the following problem:

An employer decided choosing applicants was taking too long. Instead, they to give the applicants a mark as follows. Assume there are a applicants that applied. They get the applicants to sit in a circle and chooses a number, $n < a$, at random. Then starting with the first applicant, they count the applicants clockwise until they hit n. That applicant is 'eliminated' and receives a grade of $\frac{1}{a} \cdot 100\%$. Starting from the person after the 'eliminated' applicant, they repeat the process. The next applicant to be 'eliminated' will receive a grade of $\frac{2}{a} \cdot 100\%$. After the process continues, the last applicant remaining will receive a grade of 100%.

You may see how the process works in this video, with 10 applicants and $n = 4$

Given the set of applicants, $X_1, X_2, ...X_a$, we want to be able to determine what each of their grades will be.

Write a method called ***void runGrading(int a, int n)*** that takes 2 numbers, a and n, which uses a Queue to record the order in which applicants receive their grades. Write a second method, ***getGrading*** which first calls ***runGrading*** and then returns an array of ApplicantGrades:

```
1  import java.util.LinkedList;
2
3  public class QuickGrader{
4      private Queue<Applicant> queue;
5      private Applicant[] applicants;
6      private int numberApplicants;
7      private int skip;
8
9
10     public static void main(String[] args){
11         //Example of the program running
```

```
12          QuickGrader quickGrader = new QuickGrader(10, 3);
13          Applicant[] applicants = quickGrader.getGrading();
14          for(Applicant applicant : applicants){
15              System.out.println(applicant);
16          }
17      }
18
19      public QuickGrader(int numberApplicants, int skip){
20          this.numberApplicants = numberApplicants;
21          this.applicants = new Applicant[numberApplicants];
22          this.skip = skip;
23          this.queue = new LinkedList<>();
24          initApplicants();
25      }
26
27      private void initApplicants(){
28          for(int i = 0; i < numberApplicants; i++){
29              applicants[i] = new Applicant(i, 0);
30          }
31      }
32
33      private void runGrading(){
34          // We can  use our 'circular queue' concept to implement this algorithm.
35
36          // We want to iteratively count the applicants through `skip' steps.
37
38          // We do this by removing a applicant from the front of the queue and putting them to the
            ↪  back
39
40          // Once we've hit the `skip' steps, we remove the applicant from the queue and assign them
            ↪  their grade
41
42          // We repeat this until there are no more applicants left in the queue
43
44          // To visualize this, think of yourself stay fixed in one position and rotating the queue
            ↪  around you `skip' times. Then remove whoever is in front of you and repeat.
45      }
46
47      public Applicant[] getGrading(){
48          runGrading();
49          return applicants;
50      }
51  }
52
53
54  public class Applicant{
55      int applicantNumber;
```

```
56      float grade;
57      public Applicant(int applicantNumber, int grade){
58          this.applicantNumber = applicantNumber;
59          this.grade = grade;
60      }
61
62      public String toString(){
63          return "Applicant " + applicantNumber + " got a grade of " + grade;
64      }
65
66      private void validateGrade(float grade) throws IllegalArgumentException{
67          if(grade < 0 || grade > 100){
68              throw new IllegalArgumentException("grade must be between 0 and 100");
69          }
70      }
71
72      public void setGrade(float grade){
73          validateGrade(grade);
74          this.grade = grade;
75      }
76  }
```

1. For $a = 12$ applicants and $n = 3$, determine the grades for each of the applicants.
2. For $a = 180$ and $n = 37$, in what position should you sit to receive a 50% grade?
3. For $a = 1100$ and $n = 259$, in what position should you sit to get the best grade?
4. Give the Big-O runtime of this program.